Trabalho Prático: Facial Landmark Detection para Detecção de Sinais de Sonolência

Disciplina de Visão Computacional

1 Objetivo

O objetivo deste trabalho é desenvolver, treinar e avaliar um modelo de *Facial Landmark Detection* capaz de localizar pontos-chave do rosto humano em imagens.

Após o treinamento, o modelo deverá ser utilizado em uma aplicação em vídeo, com o objetivo de identificar **sinais visuais associados à sonolência**, como o fechamento prolongado dos olhos.

O trabalho busca integrar diferentes etapas de um sistema de visão computacional:

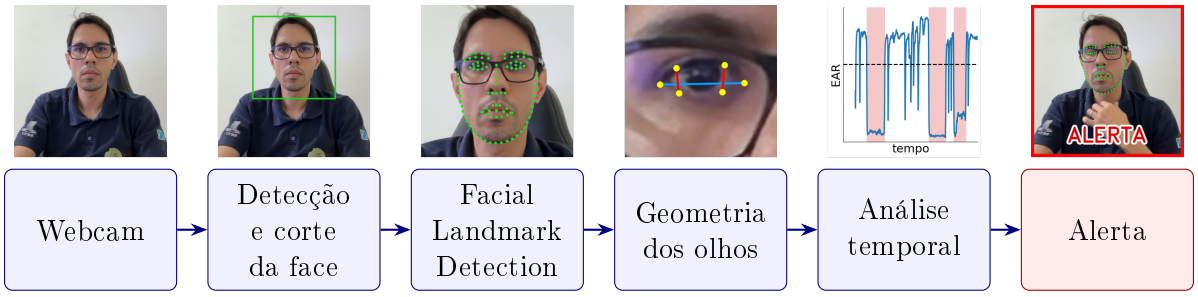
- preparação e processamento de dados;
- treinamento de redes neurais;
- estimação de pontos-chave faciais;
- avaliação quantitativa do modelo;
- inferência em imagens e vídeos;
- análise geométrica dos landmarks;
- análise temporal;
- desenvolvimento de uma aplicação utilizando webcam.

2 Descrição Geral

O trabalho será dividido em duas etapas principais.

1. Na primeira etapa, cada grupo deverá desenvolver e treinar um modelo de *Facial Landmark Detection*. O modelo deverá receber uma imagem contendo um rosto e produzir como saída a localização de diferentes pontos-chave da face.
2. Na segunda etapa, o modelo treinado deverá ser utilizado para analisar uma sequência de vídeo obtida por webcam e identificar situações em que os olhos permanecem fechados por um determinado intervalo de tempo.

O fluxo geral esperado é:



3 Materiais de Apoio

Junto com este enunciado, são fornecidos:

- `environment.yml`: ambiente *conda* com PyTorch, torchvision e OpenCV;
- `src/download_wflw.py`: download do dataset WFLW (imagens e anotações);
- `src/visualize_wflw.py`: visualização das anotações do WFLW;
- `src/verificar_entrega.py`: verificação do formato da entrega (Seção 11).

O uso desses materiais é opcional. O restante do código deverá ser desenvolvido pelo grupo.

4 Parte I — Facial Landmark Detection

4.1 Dataset

O modelo deverá ser treinado e avaliado com o dataset **WFLW (Wider Facial Landmarks in the Wild)**, que contém anotações de pontos-chave faciais em imagens capturadas em diferentes condições de pose, iluminação, expressão e oclusão. O conjunto possui 10.000 faces anotadas: 7.500 de treino e 2.500 de teste. As faces estão distribuídas em 6.551 imagens, organizadas em 60 categorias de eventos (desfiles, reuniões, esportes etc.).

Organização dos arquivos. Após o download (`src/download_wflw.py`), os arquivos ficam organizados assim:

```
data/
  WFLW_images/                          imagens (uma pasta por categoria)
    0--Parade/
    1--Handshaking/
    ...
  WFLW_annotations/
    list_98pt_rect_attr_train_test/
      list_98pt_rect_attr_train.txt     7.500 faces (treino)
      list_98pt_rect_attr_test.txt      2.500 faces (teste)
      README
    list_98pt_test/
      list_98pt_test.txt                2.500 faces (teste completo)
```

```
  list_98pt_test_largepose.txt        326 faces
  list_98pt_test_expression.txt       314 faces
  list_98pt_test_illumination.txt     698 faces
  list_98pt_test_makeup.txt           206 faces
  list_98pt_test_occlusion.txt        736 faces
  list_98pt_test_blur.txt             773 faces
  README
Mirror98.txt
```

Formato das anotações. Cada arquivo `.txt` de anotações contém **uma face por linha**, com campos separados por espaço. Nos arquivos principais de treino e de teste, que ficam na pasta `list_98pt_rect_attr_train_test/`, cada linha tem 207 campos, descritos na Tabela 1.

Campos	Quantidade	Conteúdo
1–196	196	coordenadas dos 98 landmarks: `x0 y0 x1 y1 ... x97 y97`
197–200	4	retângulo da face: `x_min y_min x_max y_max`
201–206	6	atributos: pose, expressão, iluminação, maquiagem, oclusão e desfoque (0 = normal, 1 = difícil)
207	1	caminho da imagem, relativo a `WFLW_images/`

Tabela 1: Campos de cada linha de `list_98pt_rect_attr_*.txt`.

Por exemplo, a primeira linha do arquivo de treino é, de forma abreviada:

```
309.307007 538.369019 ... 494.963222 556.534734  306 308 696 870
0 0 1 0 0 0  51--Dresses/51_Dresses_wearingdress_51_377.jpg
```

Ela descreve uma face com iluminação extrema (terceiro atributo igual a 1). O retângulo dessa face vai de $(306, 308)$ a $(696, 870)$ e o primeiro landmark está em (309,3; 538,4).

Alguns cuidados ao ler as anotações:

- **Coordenadas na imagem original.** As coordenadas estão em pixels da imagem original, que em geral contém muito mais do que a face. O recorte da face deve ser feito pelo grupo, e os landmarks devem ser transformados junto com a imagem.
- **Várias faces por imagem.** Uma mesma imagem pode aparecer em várias linhas, uma para cada face anotada; algumas imagens de multidões têm mais de 15 faces. Por isso, o exemplo de treino é a face (a linha), e não a imagem.
- **Imagens em treino e teste.** Cerca de 900 imagens têm faces no treino e outras faces no teste. A divisão oficial é por face e deve ser respeitada: não redividam o conjunto.
- **Teste por subconjuntos.** Os arquivos de `list_98pt_test/` têm apenas os landmarks e o caminho da imagem (197 campos). `list_98pt_test.txt` contém as mesmas 2.500 faces de teste. Cada um dos outros seis arquivos contém as faces de teste com o atributo correspondente igual a 1; uma face pode estar em mais de um deles. Esses subconjuntos podem ser obtidos tanto por esses arquivos quanto pelas colunas de atributos do arquivo de teste principal.

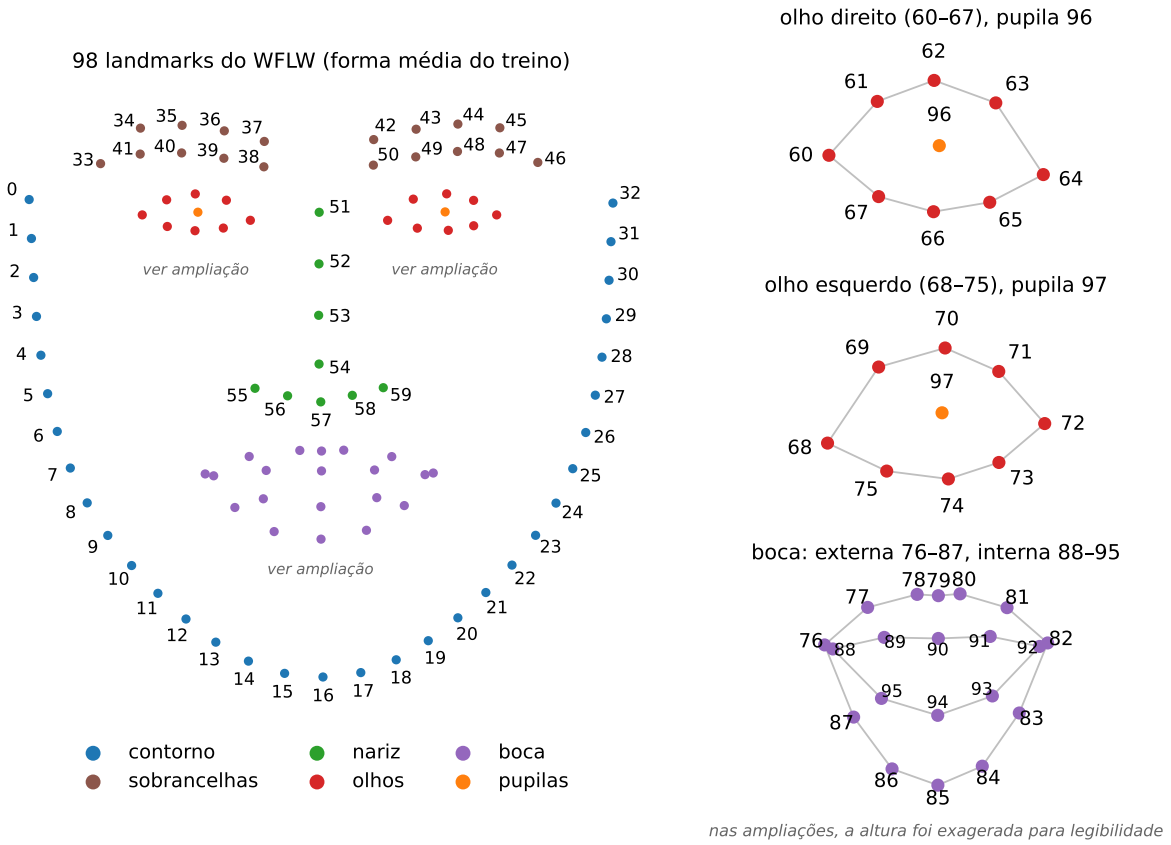


Figura 1: Numeração dos 98 landmarks do WFLW, sobre a forma média das faces de treino. À direita, ampliações dos olhos e da boca, onde os pontos ficam muito próximos na vista geral.

O WFLW utiliza **98 landmarks**, distribuídos conforme a Tabela 2 e a Figura 1. Cada olho possui oito pontos: dois cantos (60 e 64; 72 e 68), três pontos na pálpebra superior e três na inferior.

Região	Índices
Contorno facial	0–32
Sobrancelhas	33–50
Nariz	51–59
Olho direito da pessoa (à esquerda na imagem)	60–67
Olho esquerdo da pessoa (à direita na imagem)	68–75
Boca	76–95
Pupilas	96–97

Tabela 2: Índices dos landmarks do WFLW (numeração a partir de 0).

O arquivo `Mirror98.txt`, que acompanha as anotações, informa o par simétrico de cada landmark (por exemplo, o canto externo do olho direito, 60, é simétrico ao canto externo do olho esquerdo, 72). Ele é necessário para o espelhamento horizontal usado no *data augmentation* (Seção 5).

Não é obrigatório utilizar todo o conjunto de dados: a quantidade de imagens pode ser ajustada de acordo com os recursos computacionais disponíveis. Tenham em mente,

porém, que conjuntos de treino maiores e mais diversos tendem a produzir modelos mais precisos e com melhor capacidade de generalização. O relatório deverá informar claramente o tamanho do subconjunto utilizado e o critério adotado para selecioná-lo.

Atenção: olhos fechados são raros no WFLW: apenas cerca de 10% das faces de treino têm os olhos semicerrados ou fechados, e cerca de 1% têm os olhos completamente fechados. Um modelo treinado sem levar isso em conta tende a desenhar sempre um olho "médio", aberto, inclusive quando a pessoa está de olhos fechados. Isso é exatamente o que inviabiliza a Parte II. O grupo deverá verificar se o seu modelo sofre desse problema (Seção 6) e, se for o caso, propor e avaliar alguma estratégia para reduzi-lo.

Duas estratégias simples, que podem ser combinadas, são:

- **sobreamostragem dos olhos fechados**: calcular o EAR (Seção 8) com os landmarks anotados de cada face de treino e sortear com mais frequência as faces com EAR baixo (por exemplo, abaixo de 0,15), de modo que elas apareçam várias vezes por época. No PyTorch, isso pode ser feito com o `WeightedRandomSampler`, dando a essas faces um peso maior que o das demais;
- **peso maior nos landmarks dos olhos** na função de perda (Seção 4.5), para que os erros nas pálpebras pesem mais que os do contorno do rosto.

O efeito dessas estratégias aparece pouco no NME global, mas muito na medida dos olhos. Em testes com ResNet-18, remover as duas ao mesmo tempo deixou o NME global praticamente igual (até um pouco menor), mas fez a correlação entre o EAR calculado com os landmarks anotados e com os previstos cair de cerca de 0,83 para 0,51. Por isso, a avaliação deverá considerar também a medida dos olhos, e não apenas o NME.

4.2 Recorte da Face e Entrada do Modelo

As imagens do WFLW (e os frames da webcam) contêm muito mais do que a face: fundo, corpo e, muitas vezes, outras pessoas. O modelo deverá receber como entrada apenas a região da face, recortada e redimensionada para uma resolução fixa, por exemplo:

$$128 \times 128, \qquad 192 \times 192, \qquad \text{ou} \qquad 256 \times 256.$$

A resolução escolhida deverá ser informada no relatório. Resoluções maiores preservam mais detalhes dos olhos, mas deixam o treino e a inferência mais lentos.

Recorte no treinamento. No treinamento, a posição da face é conhecida pelas anotações. Recomenda-se usar o **retângulo anotado** de cada face (campos 197 a 200, Tabela 1), com a regra:

1. tomar o retângulo anotado da face;
2. construir um quadrado com o mesmo centro e lado igual ao maior lado do retângulo multiplicado por uma margem (por exemplo, 1,25, ou seja, 25% a mais), para não cortar o contorno do rosto;
3. recortar esse quadrado e redimensioná-lo para a resolução de entrada;
4. aplicar aos landmarks a mesma transformação aplicada à imagem.

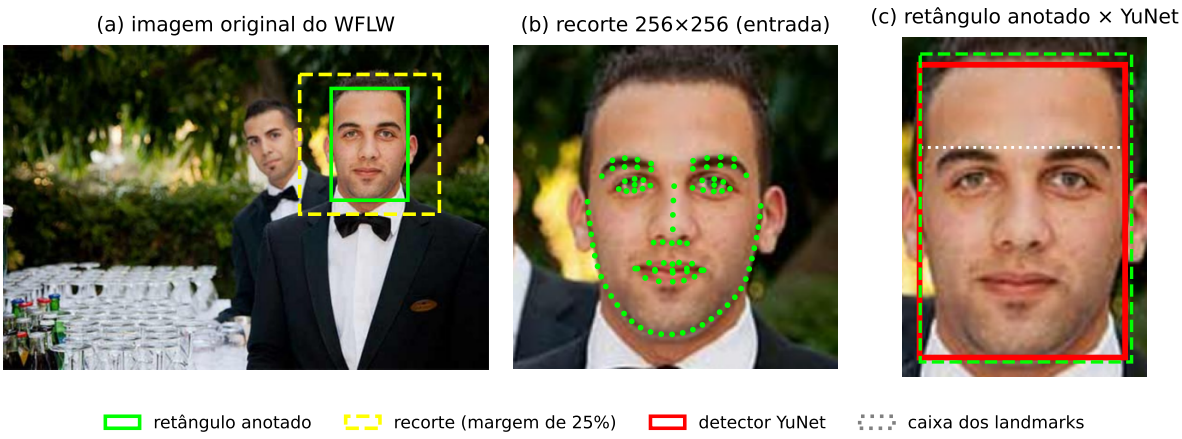


Figura 2: Recorte da face em uma imagem de teste do WFLW. (a) Retângulo anotado (verde) e recorte quadrado com margem de 25% (amarelo). (b) Recorte redimensionado para 256×256, com os landmarks transformados. (c) Retângulo anotado (verde tracejado) e caixa do detector YuNet (vermelho): as duas praticamente coincidem, pois seguem a mesma convenção e incluem a testa. Para comparação, a caixa que envolve os landmarks (pontilhada) começa bem mais abaixo.

A Figura 2 mostra um exemplo. Na imagem original (1024 × 683 pixels), o retângulo anotado da face em destaque vai de $x = 692$ a 854 e de $y = 90$ a 326 (162 × 236 pixels), com centro em $\mathbf{c} = (773;\ 208)$. Com margem de 25%, o recorte é um quadrado de lado $L = 1{,}25 \times 236 = 295$ pixels. Para uma entrada de 256 × 256, a escala é $s = 256/295 \approx 0{,}868$, e cada landmark $\mathbf{p}$ da imagem original vai para

$$\mathbf{p}' = s\,(\mathbf{p} - \mathbf{c}) + (128,\ 128).$$

O landmark 60, por exemplo, está em (720,6; 190,7) na imagem original e passa para (82,5; 113,0) no recorte. Na inferência, a transformação inversa leva as coordenadas previstas de volta para a imagem original. Com margem de 25%, todos os 98 landmarks ficam dentro do recorte em 99% das faces de treino.

As transformações geométricas do *data augmentation* (escala, rotação, translação e espelhamento) podem ser incorporadas nesse mesmo passo, variando o tamanho, o ângulo e a posição do quadrado. Variar escala e posição tem um benefício adicional: o modelo passa a tolerar recortes imperfeitos, como os que ocorrem no vídeo.

Recorte no vídeo. Na webcam não há anotações, e a caixa da face precisa ser obtida, a cada frame, com um detector de faces. Recomenda-se o YuNet do OpenCV (Seção 7.2). O recorte segue a mesma regra do treino: quadrado com o mesmo centro da caixa e lado igual a 1,25 vezes o seu maior lado.

Isso funciona porque o retângulo anotado do WFLW e a caixa do YuNet seguem a mesma convenção (Figura 2c). Nas faces de teste do WFLW, as duas caixas têm, em média, o mesmo centro e o mesmo tamanho (diferença abaixo de 2%), com variações de 4% a 9% de uma face para outra. Assim, a caixa do detector pode ser usada diretamente, sem nenhuma correção, e as variações são cobertas pelo aumento de dados de escala e translação.

Atenção: se o modelo for treinado com um tipo de recorte e receber outro no vídeo (por

exemplo, treinado com recortes justos e usado com recortes com muito fundo, ou deslocados), o desempenho cai de forma significativa, e o problema não aparece na avaliação no WFLW. Visualizem os recortes que chegam ao modelo durante a aplicação e comparem com os do treino.

O grupo deverá descrever no relatório:

- como o recorte é definido no treinamento;
- como o recorte é obtido no vídeo, e por que ele é compatível com o do treinamento.

4.3 Modelo

O grupo deverá implementar e treinar um modelo capaz de estimar os landmarks faciais.

É permitido utilizar uma rede pré-treinada para extração de características, como: ResNet-18, MobileNet, EfficientNet ou outra arquitetura equivalente.

Entretanto, a parte responsável pela localização dos landmarks deverá ser projetada e treinada pelo próprio grupo. Na prática, o modelo terá duas partes:

- **Backbone** (extrator de características): uma rede de classificação pré-treinada em uma base genérica, como a ImageNet, sem as camadas finais de classificação (*pooling* global e camada totalmente conectada). O grupo pode mantê-la congelada ou ajustar seus pesos durante o treino (*fine-tuning*), e deverá informar qual opção usou.
- **Cabeça de landmarks**: as camadas que transformam as características do backbone na saída do modelo, sejam heatmaps ou coordenadas (Seção 4.4). Essas camadas deverão ser definidas pelo grupo e treinadas a partir de pesos aleatórios, com o WFLW.

Por exemplo, a ResNet-18 sem as camadas de classificação transforma uma entrada de $256 \times 256 \times 3$ em um mapa de características de $8 \times 8 \times 512$. A partir dele, duas cabeças possíveis são:

- **para heatmaps**: algumas camadas de convolução transposta, cada uma dobrando a resolução ($8 \to 16 \to 32 \to 64$), seguidas de uma convolução 1×1 que produz K mapas de 64×64;
- **para regressão direta**: *pooling* global ($8 \times 8 \times 512 \to 512$) seguido de uma ou mais camadas totalmente conectadas, que produzem as $2K$ coordenadas.

A Tabela 3 resume o que pode ser usado em cada parte do sistema.

O relatório deverá descrever a arquitetura completa e indicar:

- a origem dos pesos iniciais do backbone;
- quais camadas foram treinadas pelo grupo;
- o número total de parâmetros e o número de parâmetros treináveis.

Um backbone maior não garante um sistema melhor: o ganho no NME global pode ser pequeno e se concentrar em regiões que não são usadas pela aplicação, enquanto o custo de inferência aumenta. A escolha deverá considerar também a velocidade necessária para o vídeo.

Componente	Permitido?
Backbone pré-treinado em classificação genérica (ImageNet), congelado ou com *fine-tuning*	sim
Cabeça de landmarks definida e treinada pelo grupo, a partir de pesos aleatórios	obrigatório
Detector de faces pronto (recomendado: YuNet), apenas para obter a caixa da face	sim
Pesos pré-treinados em bases de landmarks faciais (WFLW, 300W, COFW etc.), mesmo que só do backbone	não
Bibliotecas ou modelos prontos de landmarks (dlib, MediaPipe Face Mesh, `face_alignment`, OpenCV Facemark etc.) no sistema entregue	não
Esses mesmos modelos apenas como comparação no relatório, fora do sistema entregue	sim (opcional)

Tabela 3: Uso de modelos e pesos pré-treinados.

4.4 Representação dos Landmarks

A saída do modelo precisa codificar a posição dos K landmarks ($K = 98$ no WFLW). Há três formas comuns de fazer isso.

Heatmaps. Cada landmark k é representado por um mapa de calor $H_k(x, y)$: uma imagem com o mesmo formato para todos os pontos, que vale 1 na posição anotada (x_k, y_k) e decai como uma Gaussiana à medida que se afasta dela:

$$H_k(x, y) = \exp\left(-\frac{(x - x_k)^2 + (y - y_k)^2}{2\sigma^2}\right).$$

A rede produz K mapas, um por canal de saída, e é treinada para reproduzir esses alvos (Figura 3).

Os heatmaps costumam ter resolução menor que a da entrada. Com entrada de 256×256 e mapas de 64×64, por exemplo, cada pixel do mapa corresponde a um bloco de 4×4 pixels do recorte (passo, ou *stride*, igual a 4). Assim, (x_k, y_k) na fórmula acima são as coordenadas do landmark divididas pelo passo, e σ também é medido em pixels do mapa. Na Figura 3b, $\sigma = 2$ equivale a 8 pixels do recorte.

O valor de σ é um compromisso:

- σ pequeno produz picos estreitos e precisos, mas que ocupam poucos pixels, o que dificulta o treino (Seção 4.5);
- σ grande facilita o treino, mas faz os picos de pontos vizinhos se sobreporem (Figura 3d).

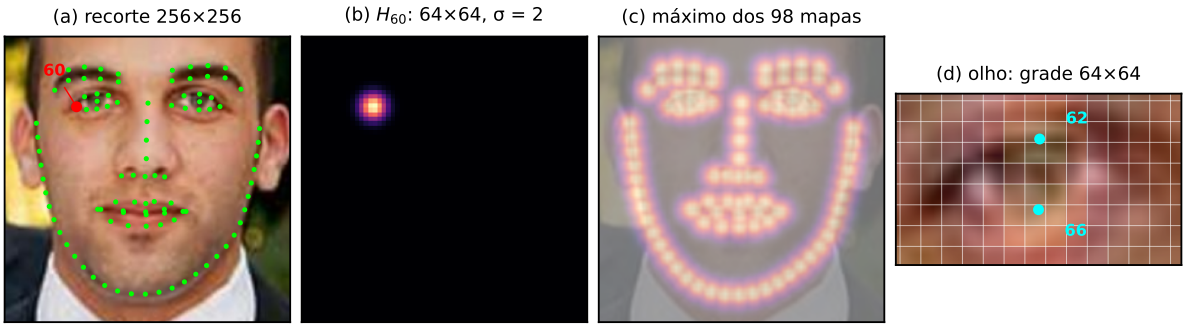


Figura 3: Heatmaps alvo para uma face do WFLW. (a) Recorte de entrada, com o landmark 60 em destaque. (b) Heatmap do landmark 60 (64×64, $\sigma = 2$). (c) Máximo dos 98 heatmaps sobreposto ao recorte; cada ponto vira uma "mancha". (d) Ampliação do olho direito com a grade do heatmap (cada célula equivale a 4×4 pixels do recorte) e os mapas dos pontos 62 (pálpebra superior) e 66 (pálpebra inferior). Com o olho aberto, os dois pontos estão a 13,6 pixels de distância, ou 3,4 células, e as Gaussianas já se sobrepõem. Com o olho semicerrado, a separação é ainda menor.

Decodificação por argmax. Para obter a posição prevista a partir de cada mapa $\hat{H}_k$ produzido pela rede, a forma mais simples é tomar o pixel de maior valor:

$$(\hat{x}_k, \hat{y}_k) = \arg\max_{x,y} \hat{H}_k(x, y),$$

e multiplicar o resultado pelo passo para voltar às coordenadas do recorte. O argmax só devolve posições inteiras do mapa. Com passo 4, isso gera um erro de quantização de até 2 pixels do recorte em cada eixo, mesmo que a rede tenha aprendido o mapa perfeitamente (Figura 4).

Regressão integral (*soft-argmax*). Em vez do pixel de maior valor, cada mapa $\hat{H}_k$ é transformado em uma distribuição de probabilidade por meio de um softmax espacial, e a posição é o valor esperado das coordenadas:

$$P_k(x, y) = \frac{e^{\hat{H}_k(x,y)}}{\sum_{x',y'} e^{\hat{H}_k(x',y')}}, \qquad \hat{x}_k = \sum_{x,y} x\, P_k(x, y), \qquad \hat{y}_k = \sum_{x,y} y\, P_k(x, y).$$

O resultado é uma média ponderada das posições do mapa, que pode cair entre dois pixels. Isso dá precisão abaixo de um pixel do mapa (Figura 4). Além disso, ao contrário do argmax, a operação é diferenciável, o que permite treinar a rede diretamente com uma perda nas coordenadas (Seção 4.5). Uma alternativa intermediária é manter o argmax e refiná-lo com uma média ponderada apenas na vizinhança do pico.

Regressão direta. A rede produz diretamente as $2K$ coordenadas, por exemplo com *pooling* global seguido de camadas totalmente conectadas. É a opção mais simples de implementar e a mais barata, mas, em geral, menos precisa que as baseadas em mapas: o *pooling* descarta a informação espacial que os heatmaps preservam.

A Tabela 4 resume as três opções. Qualquer uma delas, ou outra, poderá ser utilizada, desde que seja descrita e justificada no relatório.

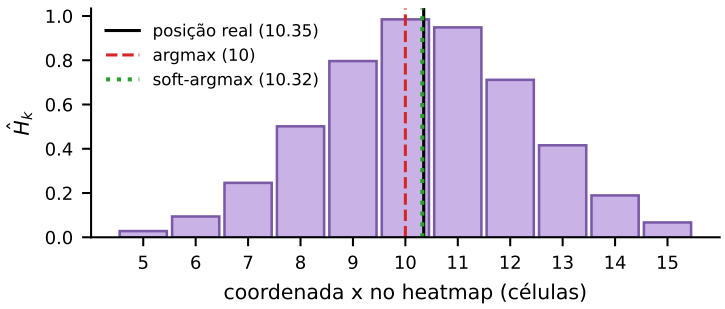


Figura 4: Perfil horizontal de um heatmap ao redor do pico. O argmax só pode devolver posições inteiras (10); o valor esperado do soft-argmax (10,32) fica próximo da posição real (10,35). Com passo 4, a diferença de 0,35 célula corresponde a 1,4 pixel do recorte.

Representação	Saída	Vantagens	Cuidados
Heatmaps + argmax	K mapas	preserva a estrutura espacial; é a abordagem clássica	quantização pelo passo; treino pode estagnar com MSE
Regressão integral	K mapas	precisão subpixel; perda direta nas coordenadas	mapas podem ficar espalhados (a confiança é menos nítida)
Regressão direta	$2K$ números	simples e rápida	em geral menos precisa

Tabela 4: Comparação entre as formas de representar os landmarks.

Atenção: a região mais importante para a Parte II é justamente a mais difícil para a resolução dos mapas. Como mostra a Figura 3d, a distância entre as pálpebras de um olho aberto é de apenas 3 a 4 células de um heatmap 64×64, e cai para 1 ou 2 com o olho semicerrado. Com o argmax, o erro de quantização é da mesma ordem que a própria abertura do olho. O grupo deverá analisar a resolução efetiva da sua representação na região dos olhos e, se necessário, utilizar alguma forma de refinamento subpixel.

4.5 Função de Perda

A função de perda depende da representação escolhida (Seção 4.4): quando a rede produz heatmaps, a perda compara mapas; quando produz coordenadas, compara posições.

Perda sobre heatmaps. A escolha mais comum é o erro quadrático médio entre os mapas previstos $\hat{H}_k$ e os alvos H_k, em todos os pixels de todos os mapas:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{KHW} \sum_{k=1}^{K} \sum_{x=1}^{W} \sum_{y=1}^{H} \left(\hat{H}_k(x,y) - H_k(x,y) \right)^2 .$$

Essa perda tem uma armadilha. Os alvos são quase inteiramente nulos: com $\sigma = 1{,}5$ em um mapa de 64×64, menos de 1% dos pixels têm valor acima de 0,1. Uma rede que prevê *zero em todos os pixels* já obtém

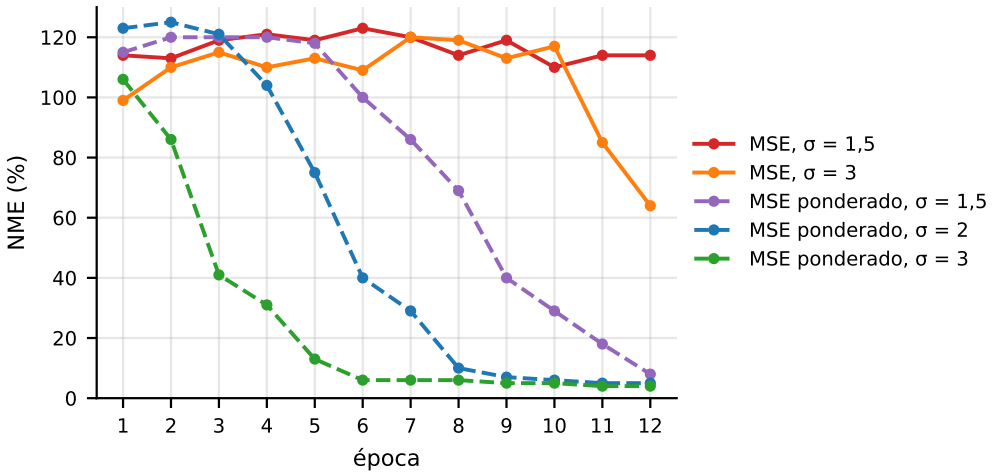


Figura 5: NME por época em experimentos com ResNet-18, heatmaps 64×64 e 2.000 faces do WFLW. Com o MSE puro (linhas contínuas), a rede fica presa prevendo mapas quase nulos, com NME acima de 100%. Com $\sigma = 3$, ela começa a sair do platô só na época 11. Com o MSE ponderado ($\alpha = 10$, linhas tracejadas), o aprendizado começa mais cedo, tanto mais cedo quanto maior o σ.

$$\mathcal{L}_{\text{MSE}} \approx \frac{\pi\sigma^2}{HW} \approx \frac{\pi \cdot 1{,}5^2}{64 \cdot 64} \approx 0{,}0017,$$

um valor baixo, sem ter aprendido nada. Na prática, a rede pode ficar presa nessa solução por muitas épocas: a perda fica estável perto desse valor e os landmarks não fazem sentido (Figura 5). Duas formas de evitar o problema são:

- **aumentar** σ, para que os picos ocupem mais pixels, ao custo de picos menos precisos e mais sobrepostos;
- **ponderar os pixels**, dando peso maior à região dos picos. Por exemplo, com um fator α (como $\alpha = 10$):

$$\mathcal{L}_{\text{MSE-pond}} = \frac{1}{KHW} \sum_{k,x,y} \big(1 + \alpha\, H_k(x,y)\big) \left(\hat{H}_k(x,y) - H_k(x,y)\right)^2.$$

Há também perdas desenvolvidas especificamente para heatmaps de landmarks, como a *Adaptive Wing Loss* [3].

Perda sobre coordenadas. Na regressão integral ou direta, a perda compara diretamente as posições previstas $\hat{\mathbf{p}}_k$ e anotadas $\mathbf{p}_k$. O erro absoluto (L1) costuma funcionar melhor que o quadrático (L2): ele penaliza menos os pontos muito errados, como os oclusos ou mal anotados, que de outra forma dominariam o gradiente. Pode-se também usar um peso w_k por landmark:

$$\mathcal{L}_{\text{coord}} = \frac{1}{K} \sum_{k=1}^{K} w_k \left\| \hat{\mathbf{p}}_k - \mathbf{p}_k \right\|_1.$$

Recomenda-se expressar as coordenadas em uma escala fixa (por exemplo, divididas pelo tamanho do recorte), para que o valor da perda não dependa da resolução de entrada. A *Wing Loss* [2] é uma alternativa ao L1 criada para landmarks faciais, que dá mais importância aos erros pequenos e médios.

Pesos por landmark. Os 98 pontos não são igualmente importantes para a aplicação. O contorno do rosto, por exemplo, é a região com maior erro, mas não é usado na Parte II. Pesos $w_k > 1$ nos landmarks dos olhos (60 a 75) fazem a rede priorizar a região que determina a medida de abertura. Esse ajuste costuma ter pouco efeito no NME global, mas pode melhorar bastante a medida dos olhos (Seção 6). Se usado, o efeito deverá ser avaliado e reportado.

Atenção: uma perda que diminui não garante que os landmarks estejam melhorando. Acompanhem o NME na validação desde as primeiras épocas, não apenas a perda, e visualizem as previsões de tempos em tempos. Se o NME não cair nas primeiras épocas, revejam a perda, σ e a decodificação antes de treinar por mais tempo.

5 Treinamento

O conjunto de teste oficial do WFLW só deverá ser utilizado na avaliação final. Para escolher hiperparâmetros e a melhor época, separe uma parte do treino como validação.

O grupo deverá documentar os principais parâmetros utilizados no treinamento, incluindo:

- arquitetura;
- resolução das imagens e critério de recorte;
- representação dos landmarks e forma de decodificação;
- número de landmarks;
- divisão entre treino, validação e teste;
- número de épocas;
- tamanho do batch;
- otimizador;
- taxa de aprendizado;
- função de perda;
- técnicas de *data augmentation*;
- estratégias de balanceamento (se houver).

São recomendadas as seguintes técnicas de aumento de dados (os valores entre parênteses são faixas que funcionaram bem em testes com o WFLW):

- espelhamento horizontal (com metade das imagens);
- pequenas rotações (até $\pm 15°$);
- mudança de escala, **aproximando e afastando** a face (fator entre 0,85 e 1,2 no lado do recorte);
- translações (até 6% do recorte em cada eixo);
- alteração de brilho e contraste.

Na mudança de escala, é importante incluir valores menores que 1, que aproximam a face e cortam parte das bordas do recorte. Em testes, usar apenas valores maiores que 1 (afastar a face) piorou o NME e a medida dos olhos. Já a oclusão artificial (cobrir partes da face com retângulos aleatórios) não trouxe ganho nos mesmos testes e pode ser omitida.

Nas transformações geométricas, as coordenadas dos landmarks deverão ser modificadas de maneira consistente com a transformação aplicada à imagem.



Figura 6: Espelhamento horizontal de uma face do treino do WFLW. Vermelho: landmarks 60–67 (olho direito da pessoa); azul: 68–75 (olho esquerdo). (a) Original. (b) Espelhando só a coordenada x, os índices 60–67 passam a marcar o olho esquerdo, e o 0 e o 32 trocam de lado. (c) Após reordenar com o `Mirror98.txt`, cada índice volta a ter o mesmo significado da imagem original.

Espelhamento horizontal e o arquivo `Mirror98.txt`

O espelhamento horizontal é uma das técnicas de aumento de dados mais úteis para faces, pois praticamente dobra a variedade do treino. Ele exige, porém, um cuidado com os landmarks: não basta espelhar as coordenadas.

Cada índice do WFLW tem um significado fixo. O índice 60, por exemplo, é sempre o canto externo do *olho direito da pessoa*, que aparece à esquerda na imagem. Ao espelhar a imagem, o olho direito passa a aparecer à direita e assume o papel de olho esquerdo. Se apenas a coordenada x for espelhada, o ponto 60 vai parar sobre o olho que agora é o esquerdo (Figura 6b). O modelo recebe, então, exemplos contraditórios: metade das imagens com o ponto 60 de um lado da face e metade do outro.

A correção é espelhar as coordenadas e depois trocar cada landmark pelo seu par simétrico (Figura 6c). O arquivo `Mirror98.txt` tem uma única linha com 98 números separados por vírgula. O número na posição i (contando a partir de 0) é o índice do par simétrico do landmark i:

```
32,31,30,...,1,0,46,45,44,...,72,71,70,69,68,75,74,73,64,...,97,96
```

Assim, mirror[0] = 32 (extremidades do contorno), mirror[60] = 72 (cantos externos dos olhos) e mirror[96] = 97 (pupilas). Pontos sobre o eixo de simetria da face, como a linha do nariz (51 a 54) e o queixo (16), são pares de si mesmos. Para uma imagem de largura W, o espelhamento completo fica:

$$x'_i = W - 1 - x_i, \qquad \mathbf{p}_i^{\text{novo}} = \mathbf{p}'_{\text{mirror}[i]}, \qquad i = 0, \ldots, 97.$$

Atenção: esse é um erro silencioso: o código roda normalmente, mas o modelo fica pior, principalmente nos olhos. Antes de treinar, visualizem algumas amostras aumentadas e confiram que os pontos 60–67 continuam no olho que aparece à esquerda da imagem.

6 Avaliação do Facial Landmark Detection

O modelo deverá ser avaliado no conjunto de teste do WFLW, utilizando o erro euclidiano normalizado:

$$NME = \frac{1}{K} \sum_{k=1}^{K} \frac{\|\hat{\mathbf{p}}_k - \mathbf{p}_k\|_2}{d},$$

onde $\hat{\mathbf{p}}_k$ é a posição estimada, $\mathbf{p}_k$ é a posição verdadeira e d é a **distância entre os cantos externos dos olhos** (landmarks 60 e 72). Essa é a normalização padrão do WFLW e deverá ser utilizada por todos os grupos, para que os resultados sejam comparáveis entre si e com a literatura.

O grupo deverá apresentar pelo menos:

- NME global no teste;
- taxa de falha: fração das faces com NME acima de 10%;
- NME por subconjunto do teste (pose, expressão, iluminação, maquiagem, oclusão e desfoque; as listas acompanham as anotações do WFLW);
- NME por região da face;
- exemplos qualitativos de boas estimativas e de falhas, com os landmarks estimados e anotados sobre as imagens.

Impacto dos landmarks na medida dos olhos. Além do NME, o grupo deverá calcular a medida de abertura dos olhos (Seção 8) com os landmarks anotados e com os landmarks estimados, para as faces do teste, e apresentar:

- um gráfico de dispersão entre as duas medidas;
- a correlação e o erro absoluto médio entre elas.

Essa análise mostra se o modelo realmente distingue olhos abertos de fechados. Um NME global baixo não garante isso: o modelo pode localizar bem o olho como um todo e, ainda assim, prever sempre a mesma abertura.

Como referência de ordem de grandeza, um modelo com backbone ResNet-18 treinado no WFLW completo atinge NME em torno de 4,5% a 5% no teste.

Recomenda-se comparar pelo menos duas variantes do modelo, por exemplo:

- duas representações ou funções de perda;
- dois backbones;
- com e sem alguma estratégia para olhos fechados.

A comparação deverá mostrar o efeito tanto no NME quanto na medida dos olhos.

7 Parte II — Aplicação em Vídeo

Após o treinamento, o modelo deverá ser utilizado para realizar inferência em uma sequência de vídeo, seguindo o fluxo da Seção 2. O modelo de landmarks utilizado nesta etapa deverá ser o modelo treinado pelo próprio grupo.

A aplicação deverá:

1. capturar imagens utilizando uma webcam ou arquivo de vídeo;
2. localizar a região facial;
3. recortar a face e executar o modelo treinado;
4. desenhar os landmarks sobre a imagem;
5. determinar o estado dos olhos (Seção 8);
6. realizar uma análise temporal;
7. gerar um alerta visual ou sonoro quando uma condição previamente definida for detectada.

7.1 Captura do Vídeo

Com o OpenCV, a mesma interface lê tanto a webcam quanto um arquivo de vídeo. Cada frame é uma matriz `numpy` de dimensões altura × largura × 3, com os canais na ordem **BGR**, e não RGB:

```
import cv2

cap = cv2.VideoCapture(0)                # webcam 0; ou "video.mp4"
fps = cap.get(cv2.CAP_PROP_FPS)          # taxa de quadros nominal
while True:
    ok, frame = cap.read()               # frame: (altura, largura, 3), BGR
    if not ok:
        break                            # fim do vídeo ou falha da câmera
    ...                                  # processamento do frame
    cv2.imshow("app", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
```

Se o modelo foi treinado com imagens RGB (por exemplo, lidas pelo PIL ou convertidas no treino), o frame precisa ser convertido antes da inferência, com `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`. Esquecer essa conversão não gera erro, mas troca os canais vermelho e azul e prejudica o modelo.

7.2 Detecção da Face com o YuNet

Para localizar a face em cada frame, recomenda-se o detector de faces **YuNet**, disponível no OpenCV (`cv2.FaceDetectorYN`). Ele é uma rede neural convolucional pequena (cerca de 230 KB), treinada para detectar faces em diferentes poses. O detector fornece apenas a caixa da face e não substitui o modelo de landmarks do grupo.

O arquivo do modelo é distribuído no repositório `opencv_zoo` e deverá ser baixado uma vez, por exemplo com:

```
wget https://github.com/opencv/opencv_zoo/raw/main/models/\
face_detection_yunet/face_detection_yunet_2023mar.onnx
```

Essa versão do modelo exige o OpenCV 4.8 ou mais recente; o ambiente fornecido já atende a esse requisito. O programa abaixo abre a webcam, detecta a face em cada frame e desenha a caixa, os 5 pontos e a confiança da detecção (tecla `q` encerra):

```
import cv2

detector = cv2.FaceDetectorYN.create(
    "face_detection_yunet_2023mar.onnx", "",
    (320, 320),             # tamanho da entrada (ajustado no laço)
    score_threshold=0.7,    # confiança mínima para aceitar uma face
    nms_threshold=0.3)      # remove caixas sobrepostas da mesma face

cap = cv2.VideoCapture(0)   # webcam 0; ou o caminho de um arquivo de vídeo
while True:
    ok, frame = cap.read()  # frame BGR, como lido pelo OpenCV
    if not ok:
        break
    h, w = frame.shape[:2]
    detector.setInputSize((w, h))       # deve coincidir com o tamanho do frame
    _, faces = detector.detect(frame)

    if faces is not None:               # None quando não há nenhuma face
        face = max(faces, key=lambda f: f[2] * f[3])   # maior face
        x, y, largura, altura = face[:4].astype(int)   # caixa da face
        pontos = face[4:14].reshape(5, 2)              # 5 pontos (x, y)
        score = face[14]                               # confiança (0 a 1)

        cv2.rectangle(frame, (x, y), (x + largura, y + altura), (0, 255, 0), 2)
        for px, py in pontos.astype(int):
            cv2.circle(frame, (px, py), 3, (0, 0, 255), -1)
        cv2.putText(frame, f"{score:.2f}", (x, y - 8),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

    cv2.imshow("YuNet", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):   # q encerra
        break

cap.release()
cv2.destroyAllWindows()
```

Cada linha de `faces` descreve uma face, com 15 valores:

Colunas	Conteúdo
0–3	caixa: canto superior esquerdo (x, y), largura e altura
4–5	olho direito da pessoa (à esquerda na imagem)
6–7	olho esquerdo da pessoa (à direita na imagem)
8–9	ponta do nariz
10–11	canto direito da boca da pessoa
12–13	canto esquerdo da boca da pessoa
14	confiança da detecção

Os principais parâmetros são:

- **tamanho da entrada** (`setInputSize`): deve ser igual ao tamanho da imagem passada a `detect`. Para acelerar, o frame pode ser reduzido antes da detecção (por exemplo, para 640 pixels de largura), multiplicando depois as coordenadas pelo fator inverso. Nos testes da disciplina, com essa redução, a detecção levou cerca de 20 ms por frame em CPU;
- `score_threshold`: valores maiores reduzem os falsos positivos, mas podem perder faces difíceis;
- `nms_threshold`: controla a eliminação de caixas sobrepostas que correspondem à mesma face.

No conjunto de teste do WFLW, o YuNet encontrou 97% das faces anotadas, incluindo 96% das faces com pose extrema e 95% das faces com oclusão (Figura 7). Ainda assim, ele pode falhar em situações extremas, como rosto muito pequeno, muito escuro ou parcialmente fora da imagem. A aplicação deverá tratar os frames em que nenhuma face é encontrada (por exemplo, mantendo o último estado conhecido e indicando na tela que a face não foi detectada).

Os 5 pontos do YuNet podem ajudar a verificar ou alinhar o recorte, mas são poucos e imprecisos para medir a abertura dos olhos. Essa medida deverá usar os landmarks do modelo do grupo.

7.3 Inferência e Visualização

Para cada frame, a face é recortada com a mesma regra do treino, convertida para o formato de entrada do modelo (resolução, ordem dos canais e normalização) e processada pela rede. As coordenadas previstas estão no sistema do recorte e precisam ser levadas de volta para o frame pela transformação inversa (Seção 4.2) antes de serem desenhadas, por exemplo com `cv2.circle`.

Além dos landmarks, a tela deverá mostrar, no mínimo:

- a medida de abertura dos olhos;
- o estado estimado (aberto ou fechado);
- o alerta, quando ativo, de forma bem visível (por exemplo, uma borda vermelha e um texto na tela).

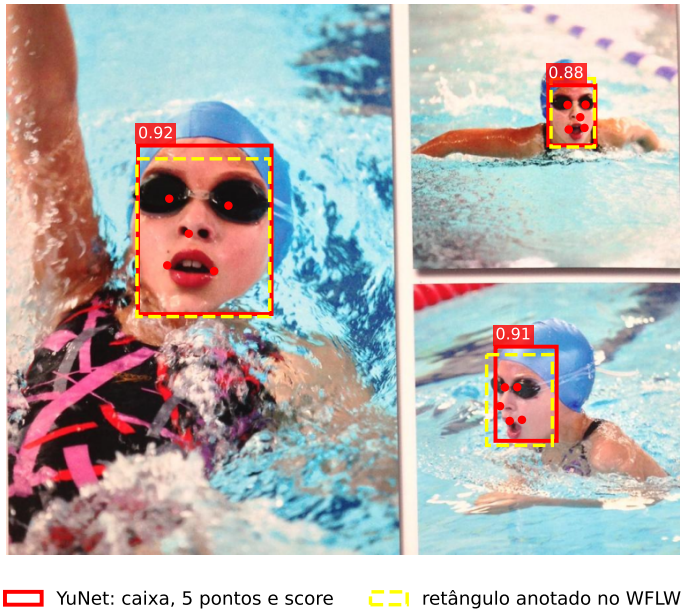


Figura 7: Detector YuNet (`score_threshold=0.7`) em uma imagem do WFLW. Ele encontra as três nadadoras, inclusive a que está de perfil, e devolve para cada uma a caixa, os 5 pontos e a confiança. As caixas do YuNet (vermelho) praticamente coincidem com os retângulos anotados no WFLW (amarelo) e podem ser usadas diretamente no recorte (Seção 4.2).

Para a inferência, coloquem o modelo em modo de avaliação (`model.eval()`) e desliguem o cálculo de gradientes (`torch.no_grad()`). Sem isso, as camadas de *batch normalization* usam estatísticas do próprio frame, e o desempenho cai.

8 Detecção do Estado dos Olhos

A partir dos landmarks localizados ao redor dos olhos, o grupo deverá definir uma medida geométrica capaz de representar o grau de abertura dos olhos.

8.1 Eye Aspect Ratio (EAR)

Uma possibilidade é utilizar o *Eye Aspect Ratio* (EAR) [1], calculado a partir de seis pontos ao redor de cada olho (Figura 8a):

- p_1 e p_4: os dois cantos do olho;
- p_2 e p_3: dois pontos na pálpebra superior;
- p_6 e p_5: os dois pontos da pálpebra inferior logo abaixo de p_2 e p_3, respectivamente.

O EAR é a razão entre a altura média do olho e a sua largura:

$$EAR = \frac{\|p_2 - p_6\|_2 + \|p_3 - p_5\|_2}{2\|p_1 - p_4\|_2}.$$

O numerador soma as duas alturas (linhas vermelhas na Figura 8), e o fator 2 no denominador faz da fração a média dessas alturas dividida pela largura (linha azul).

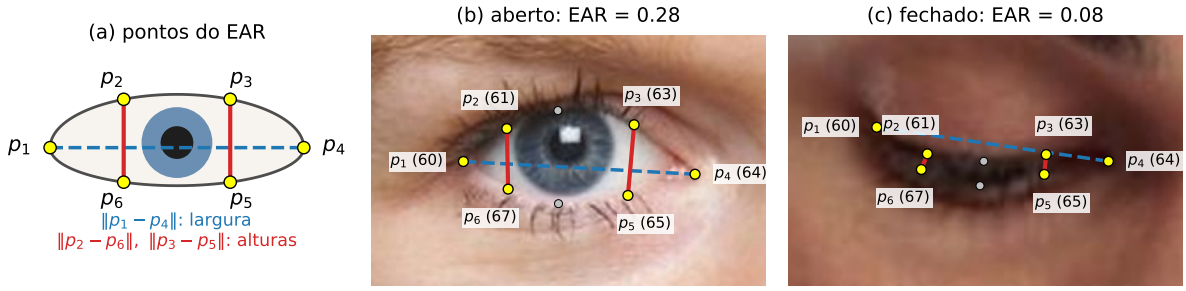


Figura 8: (a) Os seis pontos do EAR: cantos (p_1, p_4), pálpebra superior (p_2, p_3) e inferior (p_6, p_5); em azul, a largura e, em vermelho, as duas alturas. (b) e (c) Olho direito de duas faces do WFLW, com os landmarks anotados. Entre parênteses, o índice do WFLW usado para cada p_i neste exemplo; em cinza, os pontos centrais das pálpebras (62 e 66), que não foram usados. Com o olho aberto, o EAR é 0,28; com o olho fechado, cai para 0,08.

Quando o olho está aberto, as alturas são relativamente grandes; quando ele fecha, as pálpebras se encontram e as alturas tendem a zero, enquanto a largura praticamente não muda.

Por ser uma razão entre distâncias, o EAR não tem unidade e não depende do tamanho da face na imagem: ele tem o mesmo valor com a pessoa perto ou longe da câmera. Também não muda com translações nem com rotações da cabeça no plano da imagem (inclinar a cabeça para o lado).

8.2 Escolha dos Pontos no WFLW

O EAR foi proposto para modelos com seis pontos por olho. O WFLW tem oito: dois cantos, três pontos na pálpebra superior e três na inferior (Figura 1). Por isso, é preciso escolher quais pontos correspondem a $p_1, \ldots, p_6$. No exemplo da Figura 8, para o olho direito da pessoa, foram usados:

$$(p_1, p_2, p_3, p_4, p_5, p_6) = (60, 61, 63, 64, 65, 67),$$

ou seja, os cantos e os pontos laterais das pálpebras, deixando de fora os pontos centrais (62 e 66). Para o olho esquerdo, a escolha equivalente, obtida por simetria com o `Mirror98.txt`, é $(72, 71, 69, 68, 75, 73)$. Outras escolhas são possíveis. Por exemplo, usar os três pares verticais:

$$EAR_8 = \frac{\|p_{61} - p_{67}\|_2 + \|p_{62} - p_{66}\|_2 + \|p_{63} - p_{65}\|_2}{3\,\|p_{60} - p_{64}\|_2},$$

em que p_k denota o landmark k do WFLW. Usar mais pares tende a reduzir o ruído da medida.

Também é preciso decidir como combinar os dois olhos: usar a média dos dois (que reduz o ruído), o menor valor ou apenas um dos olhos. O grupo deverá documentar os pontos escolhidos e a forma de combinação, e justificar a escolha.

8.3 Valores Típicos e Limitações

Na Figura 8, calculado com os landmarks anotados, o EAR vale 0,28 com o olho aberto e 0,08 com o olho fechado. Ele não chega a zero porque as pálpebras fechadas formam uma curva, e os pontos das duas pálpebras não coincidem exatamente. Em vídeos de webcam, com os landmarks previstos por um modelo, valores entre 0,25 e 0,35 para olhos abertos e abaixo de 0,10 para olhos fechados são comuns, mas esses valores variam de pessoa para pessoa.

O EAR também diminui em situações em que o olho *não* está fechado, por exemplo:

- ao olhar para baixo, quando a pálpebra superior desce;
- ao sorrir ou apertar os olhos, quando a pálpebra inferior sobe;
- com a cabeça virada para o lado, quando a perspectiva altera a largura e as alturas do olho de formas diferentes.

Além disso, qualquer erro dos landmarks nas pálpebras se reflete diretamente no EAR: em uma face de 256×256 pixels, a altura de um olho aberto tem cerca de 10 pixels, e um erro de 2 ou 3 pixels já é uma fração grande dessa altura (Seção 6). Essas situações deverão ser consideradas ao testar a aplicação e na análise de erros.

8.4 Limiar de Decisão

Com a medida definida, o estado de cada frame pode ser decidido por um limiar:

$$EAR < T_{\text{olho}} \Rightarrow \text{olho possivelmente fechado.}$$

O valor de T_{olho} não deverá ser adotado arbitrariamente. O grupo deverá justificar a escolha, por exemplo com base nos valores do EAR observados em vídeos de teste com os olhos abertos e fechados, como na Figura 9.

Como o EAR com os olhos abertos varia de pessoa para pessoa e com a posição da câmera, um limiar fixo pode funcionar bem para uma pessoa e mal para outra. Uma estratégia é calibrar o limiar no início de cada vídeo: a pessoa mantém os olhos abertos por alguns segundos, mede-se o EAR típico nesse intervalo e define-se T_{olho} como uma fração dele (Figura 9). Nesse caso:

- a fração utilizada também deverá ser justificada;
- o EAR típico deverá ser estimado de forma robusta, por exemplo pela mediana, pois a pessoa pode piscar durante a calibração. Na Figura 9, há uma piscada perto de 2 s, que afetaria a média, mas praticamente não altera a mediana.

9 Análise Temporal

Uma única imagem com olhos fechados não deve ser suficiente para produzir um alerta, pois uma piscada normal também apresenta esse comportamento.

O sistema deverá, portanto, analisar uma sequência temporal.

Uma estratégia simples consiste em contar o número de frames consecutivos nos quais:

$$EAR < T_{\text{olho}}.$$

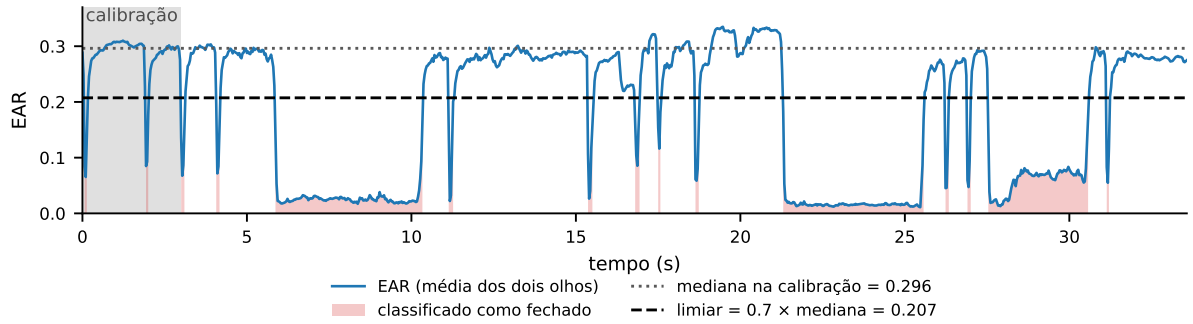


Figura 9: EAR ao longo de um vídeo de 34 s gravado com webcam, com os landmarks previstos por um modelo treinado no WFLW. Nos primeiros 3 s (em cinza), a mediana do EAR é 0,296, e o limiar é definido como 70% desse valor (0,207). Com os olhos abertos, o EAR fica em torno de 0,28; nos fechamentos, cai para perto de 0,02. Os trechos abaixo do limiar (em vermelho) incluem piscadas rápidas e três fechamentos longos.

Um alerta poderá ser produzido quando:

$$N_{\text{fechado}} > T_{\text{frames}},$$

onde N_{fechado} representa o número de frames consecutivos com os olhos classificados como fechados.

Recomenda-se, porém, trabalhar com tempo em segundos:

$$t_{\text{fechado}} > T_{\text{tempo}}.$$

Em uma webcam, a taxa de quadros real varia com a carga do computador e raramente coincide com o valor nominal da câmera. Por isso, t_{fechado} deverá ser medido pelo relógio do sistema. Em um arquivo de vídeo, o tempo deverá vir do índice do frame e da taxa de quadros do arquivo, para que o resultado não dependa da velocidade de processamento.

O grupo deverá justificar os limiares utilizados.

10 Piscadas e Fechamentos dos Olhos

O sistema deverá classificar cada fechamento dos olhos em, pelo menos, três categorias:

- **piscada**: sequência aberto → fechado → aberto de curta duração (uma piscada espontânea dura, tipicamente, entre 0,1 e 0,4 s);
- **fechamento curto**: mais longo que uma piscada, mas abaixo de T_{tempo};
- **fechamento prolongado**: permanência no estado fechado por mais de T_{tempo}, o que deverá gerar o alerta.

Os limites entre as categorias deverão ser definidos e justificados pelo grupo.

Sugere-se implementar uma pequena máquina de estados. Quando o EAR oscila perto do limiar, uma única regra "fechado se $EAR < T_{\text{olho}}$" produz várias transições seguidas e, portanto, vários eventos falsos. Duas técnicas simples reduzem esse problema:

- **histerese**: usar um limiar para fechar e outro, um pouco maior, para reabrir;

- **suavização temporal**: aplicar ao EAR, por exemplo, uma média móvel.

Ambas devem ser usadas com moderação, pois atrasam a detecção e podem apagar piscadas muito rápidas.

11 Entregáveis

A entrega deverá seguir exatamente o formato abaixo. Parte da avaliação (Seção 12) será feita por scripts que executam o modelo de cada grupo automaticamente; uma entrega fora do formato não poderá ser avaliada dessa forma.

11.1 Estrutura

Cada grupo deverá entregar um único arquivo `.zip`, nomeado com o primeiro nome de cada integrante, em letras minúsculas, sem acentos, em ordem alfabética e separados por `_`. Por exemplo, para um grupo formado por Bruno, Ana e Júlia: `ana_bruno_julia.zip`. Se dois integrantes tiverem o mesmo primeiro nome, acrescente a ambos a inicial do sobrenome (por exemplo, Ana Ferreira, Ana Souza e Bruno: `anaf_anas_bruno.zip`). O arquivo deverá conter uma pasta com o mesmo nome e a seguinte estrutura:

```
ana_bruno_julia/
  README.md                instruções de execução (ambiente, treino, aplicação)
  relatorio.pdf            relatório
  video_demo.mp4           vídeo de demonstração
  predict.py               interface do modelo
  app.py                   aplicação em vídeo
  train.py                 treinamento
  pesos/
    modelo.pt              pesos do modelo final
  modelos/                 arquivos do detector de faces (ex.: YuNet)
  requirements.txt         somente se houver dependências extras
  ...                      demais arquivos de código
```

Todo o código deverá usar caminhos relativos à pasta da entrega; nenhum caminho absoluto da máquina do grupo poderá aparecer no código.

11.2 Interface do Modelo (`predict.py`)

O arquivo `predict.py` deverá definir a classe `LandmarkModel`, com exatamente esta interface:

```python
import numpy as np


class LandmarkModel:
    def __init__(self, weights: str, device: str = "cpu"):
        """Carrega o detector de faces e o modelo de landmarks.
        weights: caminho dos pesos (pesos/modelo.pt);
        device: "cpu" ou "cuda"."""
        ...

    def predict(self, image: np.ndarray) -> np.ndarray:
        """Detecta as faces da imagem e prevê os 98 landmarks de cada uma.

        image: imagem inteira, RGB, uint8, formato (altura, largura, 3)
        retorna: array float (N, 98, 2), uma linha por face detectada,
                 com (x, y) em pixels da imagem inteira, na ordem dos
                 landmarks do WFLW; N = 0 se nenhuma face for encontrada
        """
        ...
```

O `predict` recebe apenas a imagem: a detecção das faces, o recorte, a normalização e a conversão das coordenadas de volta para a imagem inteira são responsabilidade do grupo. Assim, cada grupo pode usar o detector e a regra de recorte que escolher, e o mesmo código serve para a avaliação automática e para a aplicação em vídeo. Recomenda-se que o `app.py` use a própria classe `LandmarkModel`.

A imagem é passada em **RGB**. O OpenCV lê imagens e frames em BGR (Seção 7); no `app.py`, converta cada frame com `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` antes de chamar o `predict`. Se o detector ou o modelo do grupo trabalharem em BGR, a conversão de volta deverá ser feita dentro do `predict`.

Regras:

- o `predict` deverá funcionar em CPU, levando no máximo 2 segundos por imagem;
- o `predict` não poderá acessar a internet nem baixar arquivos: tudo o que ele usa (pesos, modelo do detector) deverá estar na entrega;
- a avaliação automática chamará o `predict` em imagens do teste do WFLW.

Associação entre faces previstas e anotadas. Uma imagem pode ter várias faces, e o `predict` devolve as faces na ordem que quiser. Na avaliação, cada face anotada é comparada com todas as faces previstas na mesma imagem e associada à de menor NME. Se nenhuma face prevista tiver NME abaixo de 50%, a face anotada conta como **não detectada**. Assim, a avaliação mede o processo completo: faces não detectadas também prejudicam o resultado.

11.3 Aplicação (`app.py`)

A aplicação deverá rodar, a partir da pasta da entrega, com:

```
python app.py --source 0                          # webcam
python app.py --source video.mp4 --out saida.mp4 --no-display
```

- `-source`: índice da webcam ou caminho de um arquivo de vídeo;
- `-out`: opcional; salva o vídeo processado, com os landmarks, a medida dos olhos, o estado estimado e o alerta desenhados;
- `-no-display`: processa sem abrir janela (para rodar em servidores).

Por padrão, a aplicação deverá carregar `pesos/modelo.pt` e o detector de faces da própria entrega, sem exigir outros argumentos. Os demais parâmetros (limiares, tempos) poderão ter argumentos opcionais, com valores padrão definidos pelo grupo.

11.4 Treinamento (`train.py`)

O treinamento deverá ser reprodutível com um único comando, que deverá estar documentado no `README.md`. Por exemplo:

```
python train.py --data caminho/para/WFLW --out pasta_de_saida
```

- o treinamento deverá usar **apenas o conjunto de treino do WFLW**, isto é, as faces do arquivo `list_98pt_rect_attr_train.txt`. O conjunto de teste e outros datasets de landmarks não poderão ser usados no treino nem na escolha de hiperparâmetros;
- a semente aleatória e os hiperparâmetros usados no modelo final deverão estar fixados no código ou no `README.md`;
- o professor poderá retreinar o modelo para conferir os resultados.

11.5 Ambiente

O código deverá rodar no ambiente fornecido (`environment.yml`). Se for indispensável algum pacote adicional, ele deverá estar listado em `requirements.txt`, instalável com `pip install -r requirements.txt`.

11.6 Relatório e Vídeo

- **relatorio.pdf**: até 10 páginas, com metodologia, experimentos, resultados e discussão, conforme as seções anteriores (incluindo a análise de erros);
- **video_demo.mp4**: até 3 minutos, gravado com a aplicação do grupo, mostrando olhos abertos, piscadas, um fechamento prolongado com o alerta e movimentos da cabeça.

11.7 Verificação Antes da Entrega

Antes de enviar, o grupo deverá rodar o verificador fornecido, a partir da raiz do projeto da disciplina:

```
python src/verificar_entrega.py caminho/para/ana_bruno_julia
```

O verificador confere os arquivos obrigatórios, o tamanho dos pesos (até 200 MB) e os argumentos do `app.py`. Em seguida, roda o `predict.py` em 100 imagens do teste do WFLW e verifica o formato da saída, o tempo por imagem, a fração de faces detectadas e o NME nas faces detectadas. A entrega só estará no formato correto se o verificador terminar com a mensagem `Entrega OK`.

12 Critérios de Avaliação

A avaliação terá uma parte automática, que mede a qualidade do modelo de landmarks, e uma parte qualitativa, que avalia o relatório e a aplicação.

Critério	Peso
Avaliação automática do modelo de landmarks	40%
Relatório (metodologia, experimentos, análise de erros e discussão)	35%
Aplicação em vídeo (vídeo de demonstração e execução do `app.py`)	25%
Total	100%

12.1 Avaliação Automática dos Landmarks

O professor executará o `predict.py` de cada grupo (Seção 11.2) em imagens do conjunto de teste do WFLW e de outros conjuntos de dados de faces anotadas. Nos conjuntos com outro esquema de anotação (por exemplo, 68 pontos), são comparados apenas os landmarks equivalentes do WFLW. Cada face anotada é associada à face prevista de menor NME, como descrito na Seção 11.2.

Em cada conjunto, são calculadas as métricas abaixo, todas com o NME normalizado pela distância entre os cantos externos dos olhos:

Métrica	Definição
Detecção	fração das faces anotadas que foram detectadas
NME	NME médio das faces detectadas, em todos os pontos comparados
Taxa de falha	fração das faces com NME acima de 10%; faces não detectadas contam como falha
NME dos olhos	NME médio das faces detectadas, apenas nos pontos dos olhos
Taxa de falha dos olhos	fração das faces com NME dos olhos acima de 10%; faces não detectadas contam como falha

As métricas dos olhos têm destaque porque são as que determinam a medida de abertura dos olhos na Parte II. A nota desta parte será calculada a partir dessas métricas, em todos os conjuntos, tendo como referência um modelo base treinado pelo professor com a mesma interface. Entregas que não passarem no verificador (Seção 11) não poderão ser avaliadas automaticamente.

12.2 Avaliação Qualitativa

- **Relatório**: clareza da metodologia e das escolhas de projeto (recorte, representação, função de perda, estratégias para os olhos), qualidade dos experimentos e da análise de erros, e a discussão da relação entre a qualidade dos landmarks, a medida dos olhos e a decisão temporal;
- **Aplicação**: funcionamento do sistema no vídeo de demonstração e na execução do `app.py` pelo professor, incluindo a distinção entre piscadas e fechamentos prolongados e o comportamento em situações difíceis (movimento da cabeça, óculos, iluminação).

Referências

[1] T. Soukupová e J. Čech. Real-Time Eye Blink Detection using Facial Landmarks. In *21st Computer Vision Winter Workshop*, 2016.

[2] Z.-H. Feng, J. Kittler, M. Awais, P. Huber e X.-J. Wu. Wing Loss for Robust Facial Landmark Localisation with Convolutional Neural Networks. In *CVPR*, 2018.

[3] X. Wang, L. Bo e L. Fuxin. Adaptive Wing Loss for Robust Face Alignment via Heatmap Regression. In *ICCV*, 2019.